

Java Lambda Expressions (Java 8)

Introduction:

- Java Lambda Expressions were introduced in **Java 8**.
- They allow writing **short and functional style code**.
- A lambda represents an **anonymous function**(no name, no class)

Key points :

- Used to implement **Functional Interface**.
- Helps in write in **clean and concise code**
- Allowing passing code as **method arguments**
- No need to create separate anonymous classes

Functional Interface:

A functional interface has only one abstract method.

Example:

```
@FunctionalInterface
interface Add{
    int addition(int a,int b);
}
```

Lambda Expression Example:

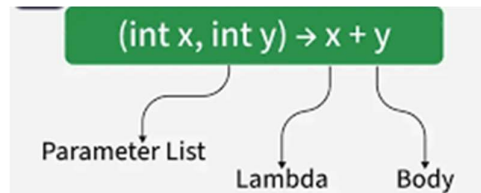
```
@FunctionalInterface
interface Add {
    int addition(int a, int b);
}

public class Main {
    public static void main(String[] args) {
        Add add = (a,b) -> a+b;
        int result = add.addition(10,20);
        System.out.println("sum: "+result);
    }
}
```

Output:

sum: 30

Syntax of lambda Expressions:



- **Parameters List** : inputs
- **Arrow Token(->)** : separates the parameters & body
- **Body** : Logic to execute

Types of Lambda parameters :

1 . Lambda with Zero Parameters

Syntax : `() -> System.out.println("Hello");`

Example:

```
@FunctionalInterface
interface ZeroParameter {
    void display();
}

public class Main {
    public static void main(String[] args) {
        ZeroParameter obj = () -> System.out.println("zero parameter Lambda");
        obj.display();
    }
}
```

Output:

zero parameter Lambda

2 . Lambda with Single Parameter

Syntax : `p -> System.out.println(p);`

Example:

```
public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> a = new ArrayList<>(Arrays.asList(1, 2, 3, 4));
        a.forEach(n->System.out.println(n));

        System.out.println("Even Elements");
        a.forEach(n->{
            if(n%2==0) System.out.println(n);
        });
    }
}
```

Output:

```
1
2
3
4
Even Elements
2
4
```

3.Lambda Expression With Multiple Parameters

Syntax :

`(p1, p2) -> System.out.println("Multiple parameters: " + p1 + ", " + p2);`

Example:

```
@FunctionalInterface
interface Functional {
    int operation(int a, int b);
}

public class Test {
    public static void main(String[] args) {

        Functional add = (a, b) -> a + b;
        Functional multiply = (a, b) -> a * b;

        System.out.println(add.operation(6, 3));
        System.out.println(multiply.operation(4, 5));
    }
}
```

Output:

9
20

Note: Lambda expressions are just like functions and they accept parameters just like functions.

Lambda in Collections / Streams :

```
import java.util.Arrays;
import java.util.List;

public class GFG{

    public static void main(String[] args){

        List<String> names = Arrays.asList(
            "Alice", "Bob", "Charlie", "Adam");

        System.out.println("All names:");
        names.forEach(name -> System.out.println(name));

        System.out.println("\nNames starting with 'A':");
        names.stream()
            .filter(n -> n.startsWith("A"))
            .map(n -> n.toUpperCase())
            .forEach(System.out::println);
    }
}
```

Output:

All names:
Alice
Bob
Charlie
Adam

Names starting with 'A':
ALICE
ADAM

Why Use Lambda Expressions?

- **Concise Code** (less boilerplate)
- Supports **Functional Programming**
- Improves **Readability**
- Powerful with **Streams & Collections**

Common Built-in Functional Interfaces:

Interface	Method	Purpose
Predicate	boolean test(T t)	Tests a given condition and returns true or false.
Consumer	void accept(T t)	Performs an action on the given argument without returning a result.
Supplier	T get()	Supplies or generates a result without taking any input.
Comparator<T>	int compare(T o1, T o2)	Compares two objects to determine their order.
Comparable<T>	int compareTo(T o)	Defines the natural ordering for objects of a class.

Validity Check Examples

Expression	Validity	Reason
() -> {}	Valid	No parameters, empty body
() -> "geeksforgeeks"	Valid	Single expression returns value
() -> { return "geeksforgeeks"; }	Valid	Uses braces with return keyword
(Integer i) -> { return "geeksforgeeks" + i; }	Valid	Correct syntax with typed parameter
(String s) -> { return "geeksforgeeks"; }	Valid	Parameter unused but valid
() -> { return "Hello" }	Invalid	Missing semicolon after return statement
x -> { return x + 1; }	Invalid	Invalid if type inference not possible
(int x, y) -> x + y	Invalid	If one parameter has type, all must